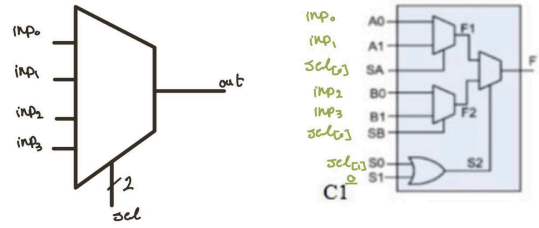
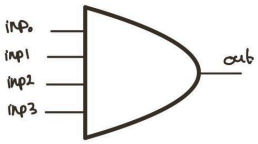
نام و نام خانوادگی: دنیز مصطفی زادگان
شماره دانشجویی: 810102603
نام و نام خانوادگی: سید حمید محمودی
شماره دانشجویی: 810102549
CAD_CA3_REPORT

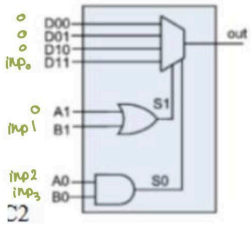
Base modules

MUX

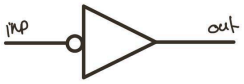


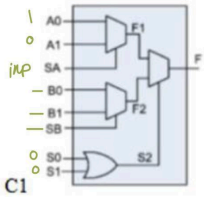
AND



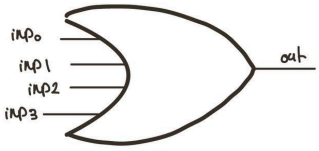


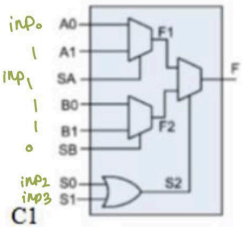
NOT



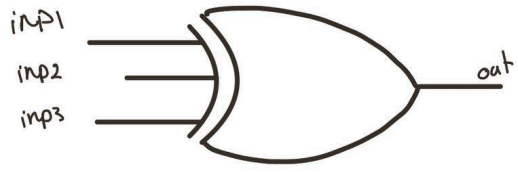


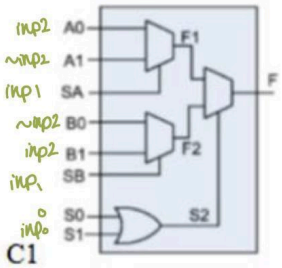
OR





XOR





Adder

```
c1 inst_carry_first(.A0(1'b0), .A1(1'B0), .SA(inp1[0]),
   .B0(1'b0), .B1(1'b1), .SB(inp1[0]),
    .S0(inp0[0]), .S1(1'b0),
     .f(c[0]));

c1 inst_sum_first(.A0(1'b0), .A1(1'b1), .SA(inp1[0]),
   .B0(1'b1), .B1(1'b0), .SB(inp1[0]),
    .S0(inp0[0]), .S1(1'b0),
     .f(out[0]));
```

$$S = A \oplus B \quad C = AB$$

چون برای محاسبه بیت اول ما Cin نداریم و نیازی به محاسبه ان نیست برای همین خارج از لوپ میذاریم

```
genvar g;
generate
    for (g = 1; g < INP_SIZE-1; g = g+1) begin : adder_gen
        c1 inst_carry(.A0(1'b0), .A1(c[g-1]), .SA(inp1[g]),
           .B0(c[g-1]), .B1(1'b1), .SB(inp1[g]),
            .S0(inp0[g]), .S1(1'b0),
             .f(c[g]));

        c1 inst_carry_inv(.A0(1'b1), .A1(1'b0), .SA(c[g-1]),
           .B0(1'b0), .B1(1'b0), .SB(1'b0),
            .S0(1'b0), .S1(1'b0),
             .f(c_inv[g-1]));

        c1 inst_sum(.A0(c[g-1]), .A1(c_inv[g-1]), .SA(inp1[g]),
           .B0(c_inv[g-1]), .B1(c[g-1]), .SB(inp1[g]),
            .S0(inp0[g]), .S1(1'b0),
             .f(out[g]));
    end
endgenerate
```

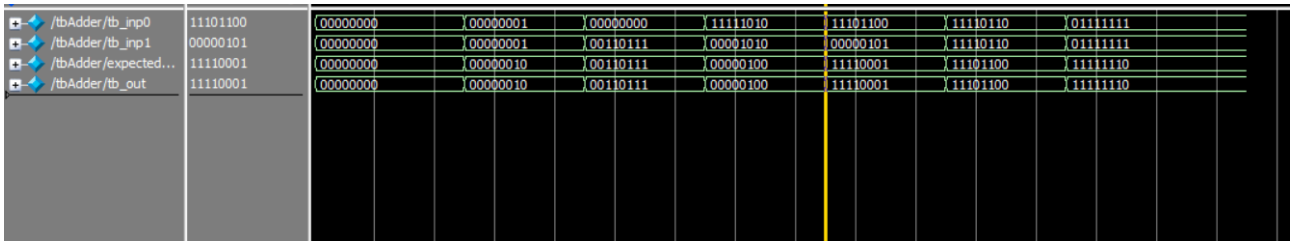
$$C = AB + AC_{in} + BC_{in} \quad S = A \oplus B \oplus C_{in}$$

```
c1 inst_carry_inv_last(.A0(1'b1), .A1(1'b0), .SA(c[INP_SIZE-2]),
   .B0(1'b0), .B1(1'b0), .SB(1'b0),
    .S0(1'b0), .S1(1'b0),
     .f(c_inv[INP_SIZE-2]));

c1 inst_sum_last(.A0(c[INP_SIZE-2]), .A1(c_inv[INP_SIZE-2]), .SA(inp1[INP_SIZE-1]),
   .B0(c_inv[INP_SIZE-2]), .B1(c[INP_SIZE-2]), .SB(inp1[INP_SIZE-1]),
    .S0(inp0[INP_SIZE-1]), .S1(1'b0),
     .f(out[INP_SIZE-1]));
```

و برای بیت اخر هم چون Cout اهمیتی ندارد نیازی به محاسبه ان نیست و ان را خارج از لوپ مینویسیم .

simulate



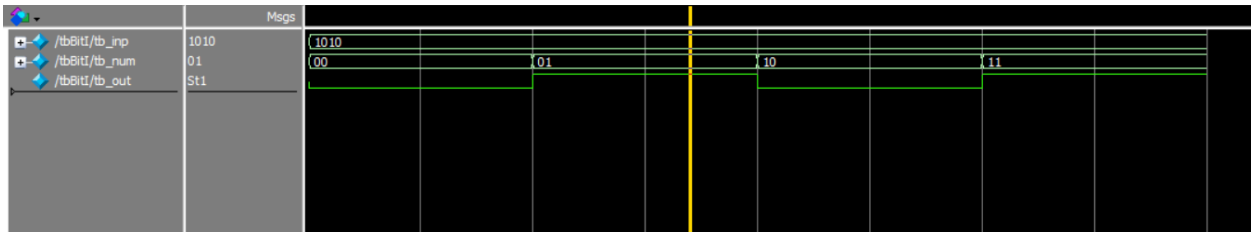

```
Time=35 |    1 +    1 =    2 | Expected:    2
Time=55 |    0 +   55 =   55 | Expected:   55
Time=75 |   -6 +   10 =    4 | Expected:    4
Time=95 |  -20 +    5 =  -15 | Expected:  -15
Time=115 |  -10 +  -10 =  -20 | Expected:  -20
Time=135 |  127 +  127 =   -2 | Expected:   -2
```

Bit_I

```
mux inst_bitI(.inp0(inp[0]), .inp1(inp[1]), .inp2(inp[2]), .inp3(inp[3]), .sel(num), .out(out));
```

با ورودی گرفتن شماره بیت i اون شماره بیت B[3:0]0 رو خروجی میدهد .
چون در ضرب کننده ورودی 4 بیتی هست ورودی رو 4 بیت در نظر گرفتیم و ورودی مولتی پلکسر قرار دادیم که با گذاشتن i روی ورودی sel همان بیت مورد نظر رو خروجی میدهد.

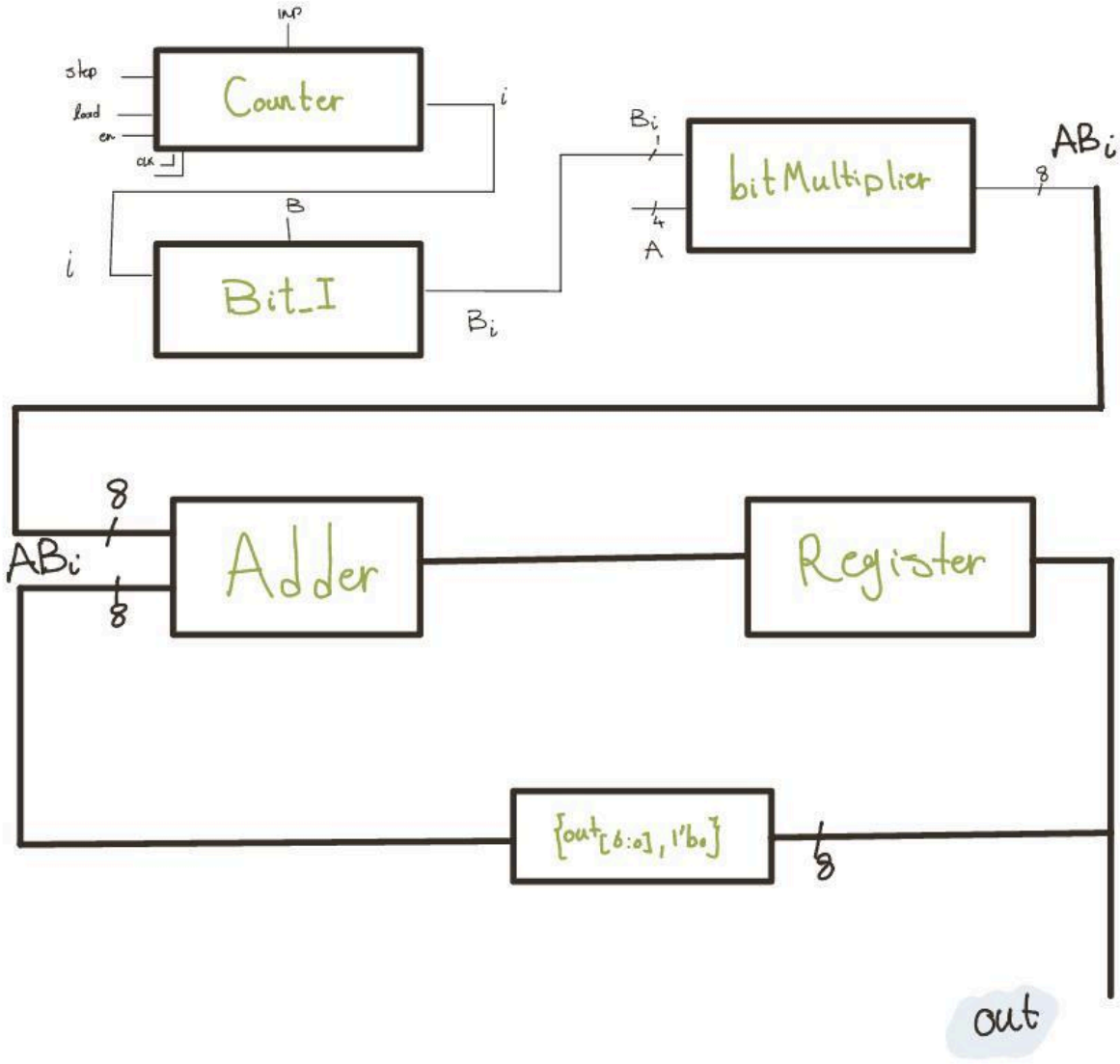
Simulate



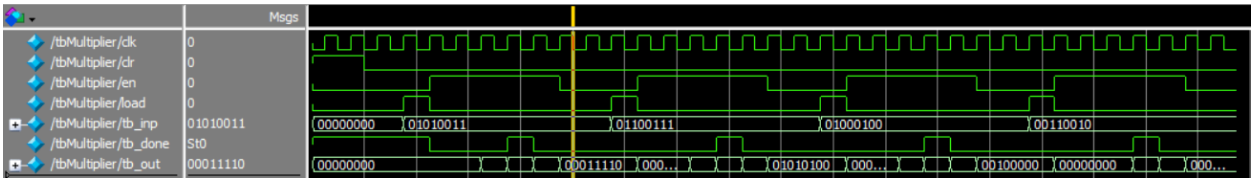

```
num=00, inp=1010, out=0 (Expected: 0)
num=01, inp=1010, out=1 (Expected: 1)
num=10, inp=1010, out=0 (Expected: 0)
num=11, inp=1010, out=1 (Expected: 1)
```

Multiplier

از الگوریتم add and shift استفاده میکنیم و از بیت 3 شروع میکنیم با counter تا بیت 0 B میشماریم و برای هر بیت حاصل ضرب را بدست اورده و در Adder با عدد قبلی جمع کرده و در reg ذخیره میکنیم سپس با assign بیت 0 را صفر میکنیم که همان عملکرد شیفت را دارد .


inp
step
load
en
clk
Counter
i
B
i
Bit_I
Bi
Bi
1
4
A
bitMultiplier
ABi
8
8
ABi
8
Adder
Register
{out[6:0], 1'b0}
8
out

simulate




```
Time=95 | Input: 3 * 5 | Output= 15
Time=175 | Input: 7 * 6 | Output= 42
Time=255 | Input: 4 * 4 | Output= 16
Time=335 | Input: 2 * 3 | Output=  6
```

Counter

```
adder #(
    .INP_SIZE(INP_SIZE)
)
counter_adder(
    .inp0(out),
    .inp1(step),
    .out(out_adder)
);

register #(
    .INP_SIZE(INP_SIZE)
)
counter_register  (
    .clk(clk),
    .clr(clr),
    .inp1(out_adder),
    .inp2(inp),
    .inp3({{INP_SIZE}{1'b0}}),
    .sel(sel_reg),
    .out(out)
);
```

Simulate

Register

```
genvar g;
generate
    for (g = 0; g < INP_SIZE; g = g+1) begin : register_gen
        s2 s2_register_inst(.D00(out[g]), .D01(inp1[g]), .D10(inp2[g]), .D11(inp3[g]),
         .A1(sel[1]), .B1(1'b0), .A0(sel[0]), .B0(1'b1),
          .clr(clr), .clk(clk),
           .out(out[g]));
    end
endgenerate
```

Simulate

```
Time=25 | Sel=1 (Load Inp1) | Inp1= 15 | Out=  0
Time=35 | Sel=2 (Load Inp2) | Inp2= 25 | Out= 15
Time=45 | Sel=3 (Load Inp3) | Inp3= 35 | Out= 25
```

CalF

```
genvar g;
generate
    for (g = 0; g < INP_SIZE; g = g+1) begin : F_gen
    c1 inst_D_inv(.A0(1'b1), .A1(1'b0), .SA(D[g]),
                  .B0(1'b0), .B1(1'b0), .SB(1'b0),
                  .S0(1'b0), .S1(1'b0),
                  .f(D_inv[g]));

    c1 inst_F0( .A0(D[g]), .A1(C[g]), .SA(B[g]),
                .B0(1'b0), .B1(1'b0), .SB(1'b0),
                .S0(1'b0), .S1(1'b0),
                .f(F0[g]));

    c1 inst_F1( .A0(C[g]), .A1(B[g]), .SA(D[g]),
                .B0(1'b0), .B1(1'b0), .SB(1'b0),
                .S0(1'b0), .S1(1'b0),
                .f(F1[g]));

    c1 inst_F2( .A0(D[g]), .A1(D_inv[g]), .SA(C[g]),
                .B0(D_inv[g]), .B1(D[g]), .SB(C[g]),
                .S0(B[g]), .S1(1'b0),
                .f(F2[g]));

    c1 inst_F3( .A0(D_inv[g]), .A1(1'b1), .SA(B[g]),
                .B0(D[g]), .B1(1'b0), .SB(B[g]),
                .S0(1'b0), .S1(C[g]),
                .f(F3[g]));

    end
endgenerate
```

F0 -> Multiplexer , sel = B
F1 -> Multiplexer , sel = D

F2 -> B = 0 / = C xor D

B = 1 / = ~(C xor D)

Simulate

```
i= 5 | B=f0 C=aa D=3c | F=ac | Exp=ac
i=20 | B=aa C=55 D=f0 | F=a5 | Exp=a5
i=40 | B=ff C=00 D=aa | F=55 | Exp=55
i=60 | B=0f C=f0 D=cc | F=cf | Exp=cf
```

Rnd_datapath

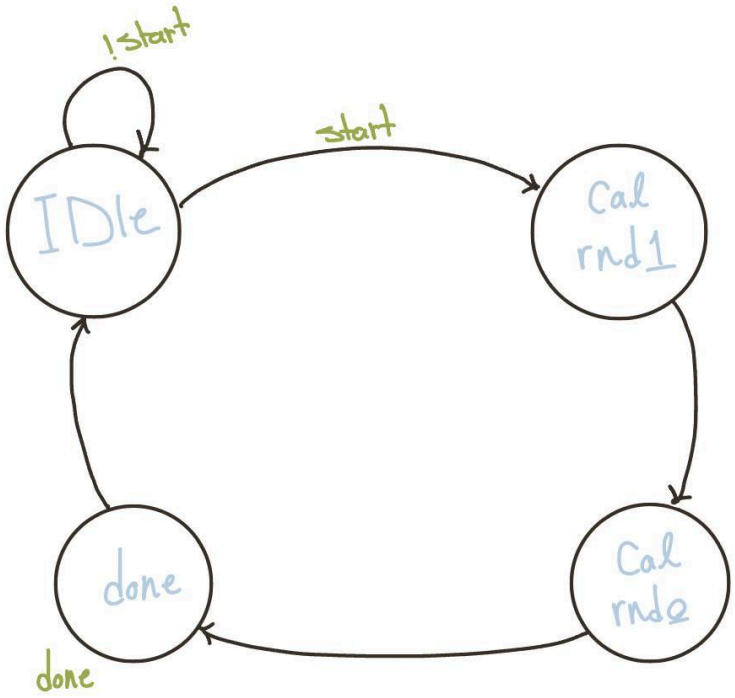
```
register #(
    .INP_SIZE(ITT_SIZE)
) x_reg (
    .clk(clk), .clr(rst),
     .inp1(itt),
     .inp2({x_reg_out[ITT_SIZE-2:0], fb}),
     .inp3({ITT_SIZE{1'b0}}),
      .sel({en, load}),
       .out(x_reg_out));

xorModule #(.INP_SIZE(1)) xor_inst_fb (.inp0(x_reg_out[1]), .inp1(x_reg_out[3]), .inp2(x_reg_out[5]), .out(fb));
```





Rnd_fsm

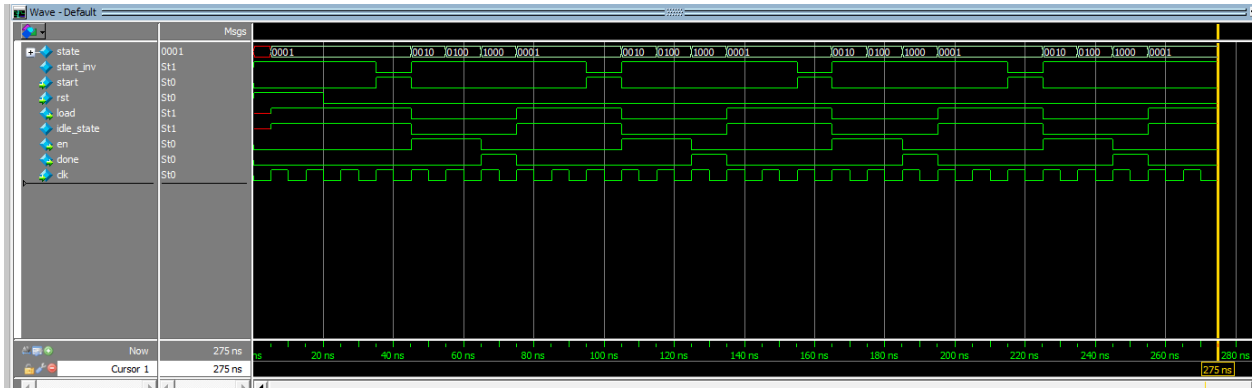


For fsm implementation we will use s2 modules so that the clr just affects the reset function and we can use other parts for logic.
So in here state idle is as followed: idle = idle.!start + done
Also this specific state requires rst to set it to 1, so we use the inputs of out s2 like a mux using rst as a selector and then give it the input of idle.!start which is made by a c2 as a secondary input next to 1.
The rest of the states get their input simply from the previous one and state cal_rnd1 get it form idle.start, also then rst is one since we connected it to clr of the rest of the states they turn 0 and state idle turns 1.

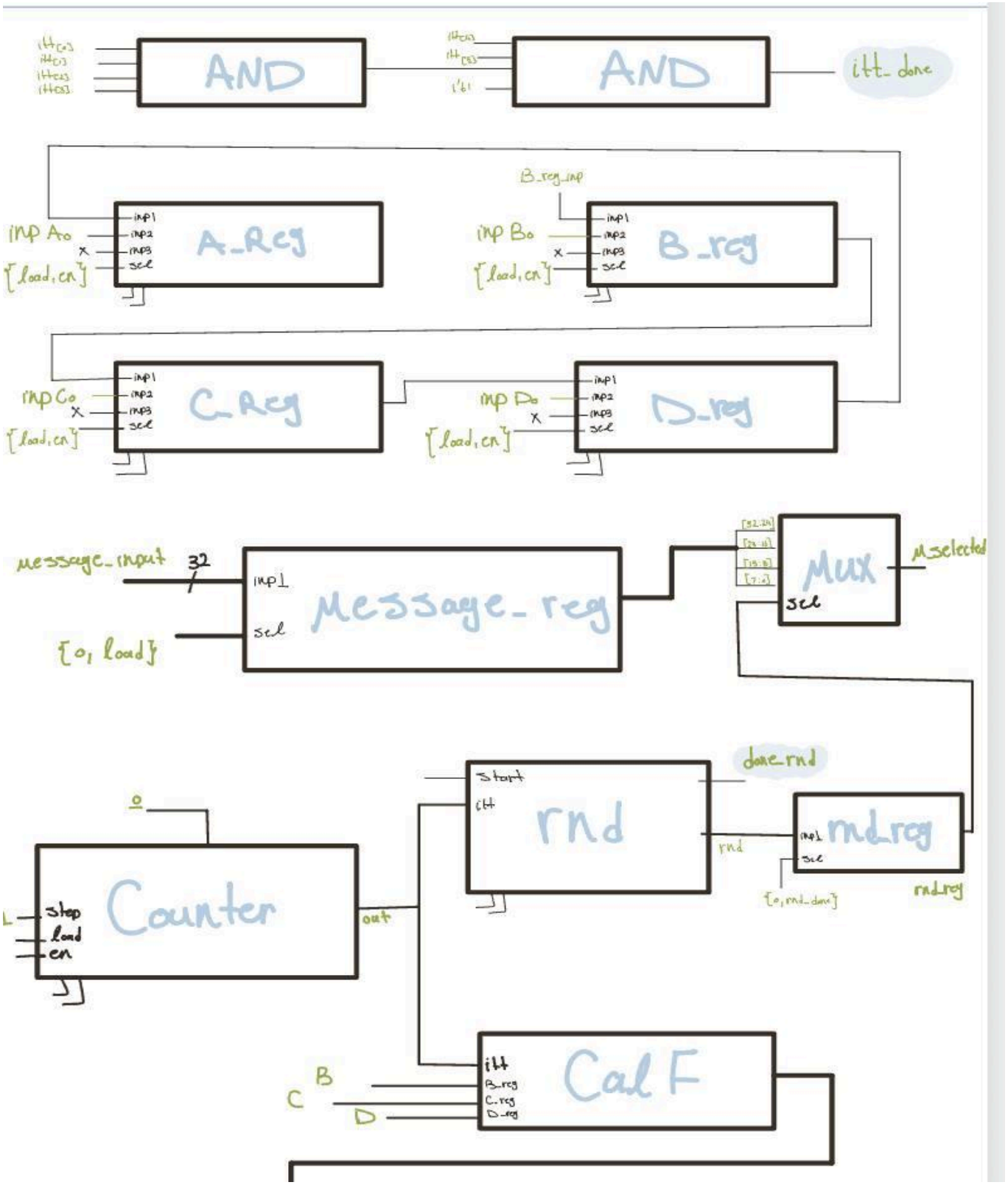
- We also need a not module which takes a c1 to make !start;

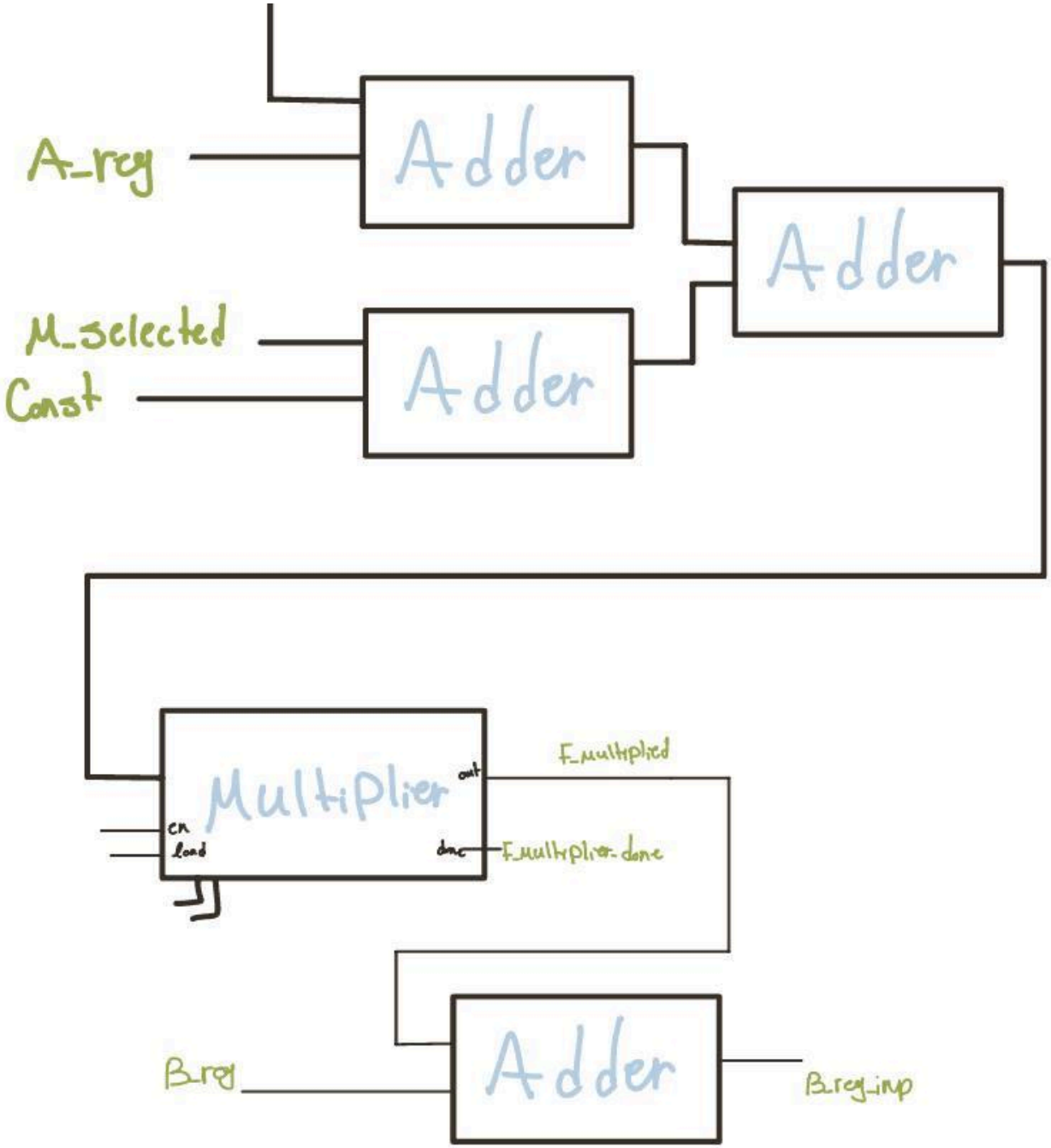
Simulate

```
Input itt=000010 | Output rnd=10
Input itt=000111 | Output rnd=10
Input itt=011101 | Output rnd=11
Input itt=110110 | Output rnd=00
```


Wave - Default
Msgs
state
start_inv
start
rst
load
idle_state
en
done
clk
0001
St1
St0
St0
St1
St1
St0
St0
St0
0001
0010
0100
1000
0001
0010
0100
1000
0001
0010
0100
1000
0001
0010
0100
1000
0001
Now
275 ns
Cursor 1
275 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns
160 ns
180 ns
200 ns
220 ns
240 ns
260 ns
280 ns
275 ns

Datapath


AND
AND
itt_done
B_reg_inp
inp Ao
A_Reg
inp Bo
B_reg
inp Co
C_Reg
inp Do
D_reg
message_input
32
Message_reg
Mux
M_selected
done_rnd
Start
itt
rnd
md_reg
rnd
rnd_reg
Counter
step
load
en
out
Cal F
B
C
D

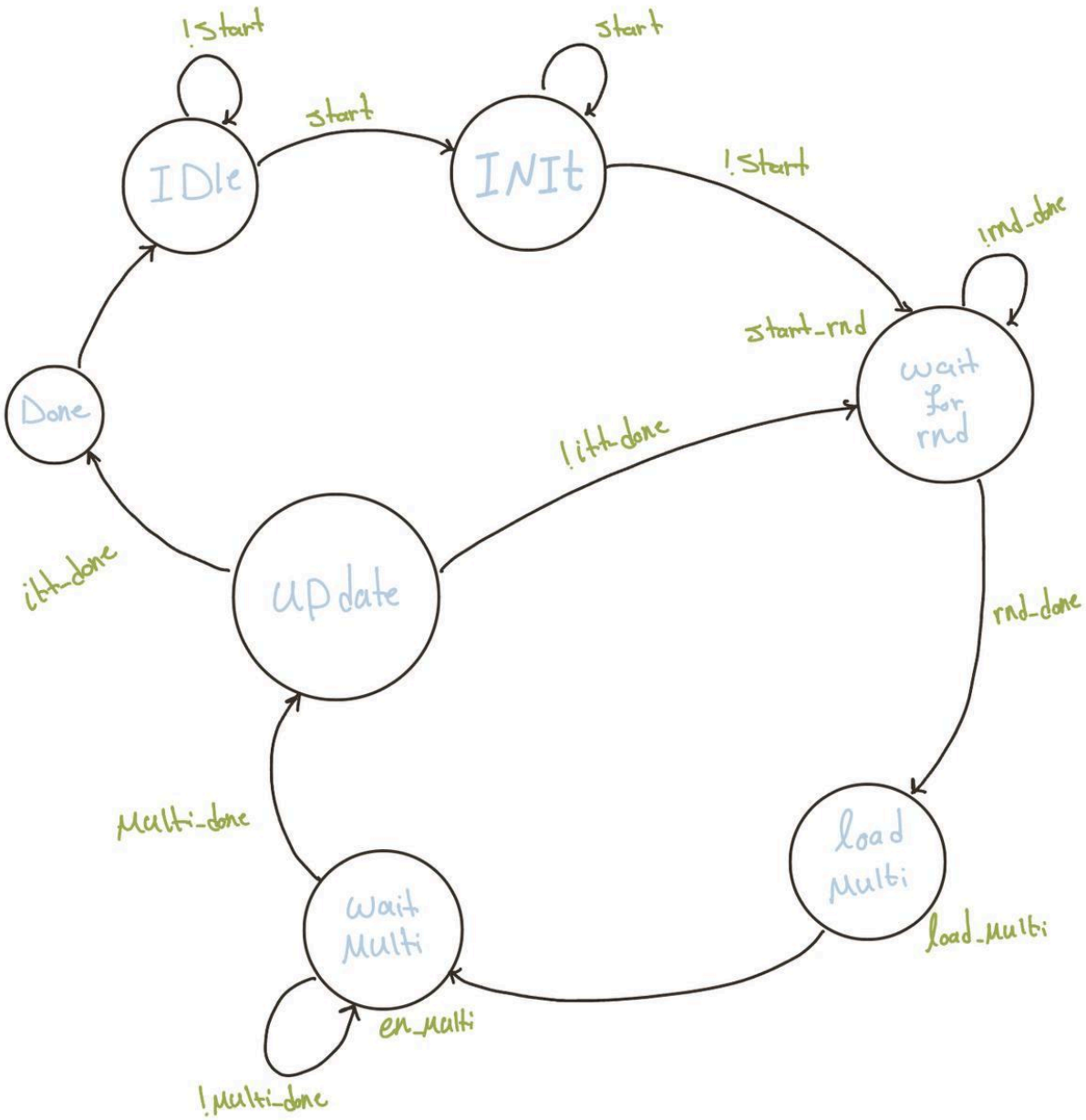

A_reg
Adder
Adder
M_selected
Const
Adder
Multiplier
out
clk
load
done
F_multiplied
F_multiplier_done
B_reg
Adder
B_reg_inp

Fsm

Here just like the previous fsm we have idle which ists clr is 0 but has its rst as a mux to get its function which is: idle = idle.!start + done
All other states are as followed and expect wair_for_rnd can be implemented using a single s2 we also need a c1 since the logic is more complex and is 3 or of 3 ands
Here are all the states:
init = idle.start + init.start
wait_for_rnd = init.!start + wait+for_rnd.!rnd_done + update.!itt_done
load_multi = wait_for_rnd.rnd_done
wait_multi = load_multi + wait_multi.!multi_done
update = wait_multi,multi_done
done = update.itt_done
Also all output signals are assign to their own correct state as followed:

```
    assign load_init = init;
    assign load_itt = init;
    assign start_rnd = wait_rnd;
    assign load_F_multiplier = load_multi;
    assign en_F_multiplier = wait_multi;
    assign en_reg = update;
    assign en_itt = update;
```

And thus this fsm is finished.

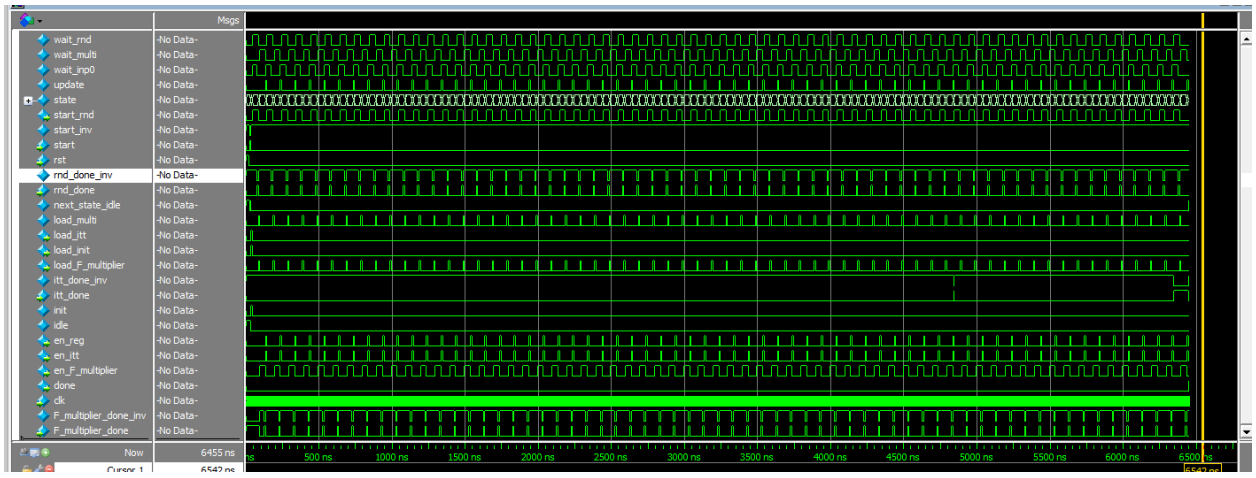

!start
start
IDle
start
INIt
!start
!rnd_done
start_rnd
wait for rnd
Done
!ith_done
ith_done
update
rnd_done
Multi_done
load Multi
wait Multi
load_multi
en_multi
!multi_done

Final simulation of 64 inputs/outputs:

Below is the displayed output of all 64 test cases we were given and is shown they are all correctly computed. (An image is large and cannot fit and also wont be clean so we have just pasted the output here)

```
# Test 1/64: SUCCESS | In: 3761eded | Out: 306749d9 | Exp: 306749d9
# Test 2/64: SUCCESS | In: df01f0b7 | Out: 405e5ecf | Exp: 405e5ecf
# Test 3/64: SUCCESS | In: 275cc00b | Out: 63045fab | Exp: 63045fab
# Test 4/64: SUCCESS | In: ac7f8ef7 | Out: 6233a418 | Exp: 6233a418
# Test 5/64: SUCCESS | In: 30d4a98b | Out: 04e5b46c | Exp: 04e5b46c
# Test 6/64: SUCCESS | In: 3e7de0a7 | Out: 08059c8c | Exp: 08059c8c
# Test 7/64: SUCCESS | In: 1cca231f | Out: dd261cdd | Exp: dd261cdd
# Test 8/64: SUCCESS | In: 14dbfde5 | Out: ce9797ee | Exp: ce9797ee
# Test 9/64: SUCCESS | In: ddbcbb86 | Out: 5902dab3 | Exp: 5902dab3
# Test 10/64: SUCCESS | In: ac390aa7 | Out: a0dbccae | Exp: a0dbccae
# Test 11/64: SUCCESS | In: 9a394a17 | Out: ff8585e0 | Exp: ff8585e0
# Test 12/64: SUCCESS | In: 45f8e80a | Out: 055d2d23 | Exp: 055d2d23
# Test 13/64: SUCCESS | In: 73b3fad7 | Out: a3390fc7 | Exp: a3390fc7
# Test 14/64: SUCCESS | In: fe011fdd | Out: 4509f57d | Exp: 4509f57d
# Test 15/64: SUCCESS | In: a2b38da0 | Out: 1b3a2b25 | Exp: 1b3a2b25
# Test 16/64: SUCCESS | In: 5539e441 | Out: a307f5f1 | Exp: a307f5f1
# Test 17/64: SUCCESS | In: da49e5c1 | Out: 5fe63e07 | Exp: 5fe63e07
# Test 18/64: SUCCESS | In: 2dfc96a1 | Out: d8e2e2e2 | Exp: d8e2e2e2
# Test 19/64: SUCCESS | In: 8c4465b2 | Out: 5b9d755b | Exp: 5b9d755b
# Test 20/64: SUCCESS | In: c5178e73 | Out: 82756fd3 | Exp: 82756fd3
# Test 21/64: SUCCESS | In: a61d71fe | Out: 2750e48a | Exp: 2750e48a
# Test 22/64: SUCCESS | In: 82690264 | Out: f1966c11 | Exp: f1966c11
# Test 23/64: SUCCESS | In: f0544526 | Out: 8afbfbf3 | Exp: 8afbfbf3
# Test 24/64: SUCCESS | In: 6813da81 | Out: 41605f41 | Exp: 41605f41
# Test 25/64: SUCCESS | In: f1db4a46 | Out: 18fe8018 | Exp: 18fe8018
# Test 26/64: SUCCESS | In: 185a1b35 | Out: 13736d6d | Exp: 13736d6d
# Test 27/64: SUCCESS | In: 031749d3 | Out: d90dd129 | Exp: d90dd129
# Test 28/64: SUCCESS | In: 3659ac4c | Out: 51907b6f | Exp: 51907b6f
# Test 29/64: SUCCESS | In: 253ac00d | Out: 8f150d0d | Exp: 8f150d0d
# Test 30/64: SUCCESS | In: e493a7d2 | Out: 8dd9d7a5 | Exp: 8dd9d7a5
# Test 31/64: SUCCESS | In: ad98a2ca | Out: 60d48c8c | Exp: 60d48c8c
# Test 32/64: SUCCESS | In: 66f66097 | Out: 5902c672 | Exp: 5902c672
# Test 33/64: SUCCESS | In: f760dc3c | Out: 52e1866a | Exp: 52e1866a
# Test 34/64: SUCCESS | In: 331141c9 | Out: a8401910 | Exp: a8401910
# Test 35/64: SUCCESS | In: 901a206b | Out: fb7f7d41 | Exp: fb7f7d41
# Test 36/64: SUCCESS | In: ed5f25f2 | Out: 3bb2ac43 | Exp: 3bb2ac43
# Test 37/64: SUCCESS | In: fc981fa1 | Out: 44eebab2 | Exp: 44eebab2
# Test 38/64: SUCCESS | In: 4d9bc43f | Out: b1e2dbc7 | Exp: b1e2dbc7
# Test 39/64: SUCCESS | In: b0144d28 | Out: 07423c2f | Exp: 07423c2f
# Test 40/64: SUCCESS | In: 0ef1eb2a | Out: 21593e7b | Exp: 21593e7b
# Test 41/64: SUCCESS | In: 0b77de8f | Out: 1a3a3224 | Exp: 1a3a3224
# Test 42/64: SUCCESS | In: b5503a5b | Out: d7ecd7d7 | Exp: d7ecd7d7
# Test 43/64: SUCCESS | In: dfaae211 | Out: 55ead1a5 | Exp: 55ead1a5
# Test 44/64: SUCCESS | In: 1bf4f083 | Out: 992b13c6 | Exp: 992b13c6
# Test 45/64: SUCCESS | In: 7e31960f | Out: 2f27a053 | Exp: 2f27a053
# Test 46/64: SUCCESS | In: eca51c62 | Out: da703434 | Exp: da703434
# Test 47/64: SUCCESS | In: 4c71ee4b | Out: 70ff5a88 | Exp: 70ff5a88
# Test 48/64: SUCCESS | In: ace6c8aa | Out: 8cfcc8b8 | Exp: 8cfcc8b8
# Test 49/64: SUCCESS | In: 6379cbe3 | Out: 4c2fd5ae | Exp: 4c2fd5ae
# Test 50/64: SUCCESS | In: 71c55fdd | Out: ce831fe3 | Exp: ce831fe3
# Test 51/64: SUCCESS | In: 4e71b366 | Out: 04c03e20 | Exp: 04c03e20
# Test 52/64: SUCCESS | In: e5b4025d | Out: 47c48255 | Exp: 47c48255
# Test 53/64: SUCCESS | In: 4c9fb025 | Out: 299e6642 | Exp: 299e6642
# Test 54/64: SUCCESS | In: ee144e06 | Out: d4734913 | Exp: d4734913
# Test 55/64: SUCCESS | In: 0c9ceadf | Out: 32a3fe7a | Exp: 32a3fe7a
# Test 56/64: SUCCESS | In: a071263d | Out: 6789b767 | Exp: 6789b767
# Test 57/64: SUCCESS | In: f41fa52d | Out: 2e996155 | Exp: 2e996155
# Test 58/64: SUCCESS | In: 78f11110 | Out: 90594de4 | Exp: 90594de4
# Test 59/64: SUCCESS | In: 6e2a1d6c | Out: a3d3c4a3 | Exp: a3d3c4a3
# Test 60/64: SUCCESS | In: 734e094d | Out: a9b0faa9 | Exp: a9b0faa9
# Test 61/64: SUCCESS | In: d6cab7ca | Out: 66e6cea6 | Exp: 66e6cea6
# Test 62/64: SUCCESS | In: 47084cb3 | Out: e74b3505 | Exp: e74b3505
# Test 63/64: SUCCESS | In: 0fb71ba6 | Out: ffd6aca4 | Exp: ffd6aca4
# Test 64/64: SUCCESS | In: feecf6da | Out: ce0efaee | Exp: ce0efaee
# Total Correct: 64 / 64
# Break in Module tbFullTop at D:/University/Term-5/CAD/CA's/CA-3/codes/tbFullTop.v line 72
```

Here a wave form of how states change: (its another test bench with just one single input for better observation)



And a closer look:

